

NiceGUI 中 app.storage 深度解析

在 NiceGUI 中，`app.storage` 是框架提供的**分层数据存储系统**，为应用提供了内存级临时存储、文件级持久化存储和应用级全局存储能力，是实现会话状态保持、用户配置持久化、全局参数共享的核心工具。它封装了底层的存储实现（内存、JSON 文件），提供了统一且简洁的键值对操作接口，完美适配 NiceGUI 的会话管理和应用生命周期。本文将从**核心定位、存储分层、使用方法、持久化机制、高级场景、注意事项**六个维度，对 `app.storage` 进行全方位的详细阐述。

一、app.storage 的核心定位

`app.storage` 是 NiceGUI 为解决**不同粒度的数据存储需求**设计的分层存储接口，其核心定位可总结为三点：

- 多粒度存储**：按**会话隔离、用户隔离、应用全局**三个维度划分存储层，满足临时状态、持久化配置、全局参数的不同存储需求；
- 统一接口**：所有存储层均实现了类似 Python 字典的键值对操作接口（`__getitem__`、`__setitem__`、`get()`、`clear()` 等），使用方式一致；
- 透明持久化**：对文件级存储（用户、全局）实现了**自动序列化 / 反序列化**，开发者无需手动处理 JSON 文件读写，数据操作与内存字典无异；
- 会话联动**：与 `ui.session` 深度集成，会话存储层直接关联客户端的会话 ID，实现客户端隔离的临时数据存储。

`app.storage` 弥补了 `ui.session` 仅支持内存临时存储的不足，同时简化了开发者对持久化数据的处理，避免了手动编写文件读写逻辑的繁琐。

二、app.storage 的存储分层

`app.storage` 包含**三个核心存储层**，分别对应不同的隔离级别、存储介质和使用场景，是理解其使用的关键。以下是各层的详细对比：

存储层	访问接口	存储介质	隔离级别	持久化特性	数据生命周期	典型使用场景
会话存储	<code>app.storage.session</code>	服务端内存	按 会话 ID 隔离（客户端隔离）	非持久化	服务重启 / 会话失效后丢失	临时会话数据（如未登录的筛选条件、表单临时输入）
用户存储	<code>app.storage.user</code>	本地 JSON 文件	按 用户标识 隔离（默认绑定会话 ID）	持久化	永久保存（除非手动删除文件 / 数据）	用户个性化配置（如主题、记住登录状态、用户偏好）
全局存储	<code>app.storage.general</code>	本地 JSON 文件	无隔离（应用级全局共享）	持久化	永久保存（除非手动删除）	应用全局参数（如版本号、系统配置、全局开关）

关键概念补充

1. **隔离级别**：指数据是否在不同客户端 / 用户间隔离。会话存储和用户存储是**客户端隔离**的，每个客户端拥有独立的数据集；全局存储是**应用共享**的，所有客户端访问同一数据集。
2. **用户标识**：`app.storage.user` 默认使用客户端的**会话 ID**作为用户标识，因此在客户端未清除 Cookie 的情况下，即使服务重启，仍可通过会话 ID 关联到原有的用户存储数据。
3. **序列化**：文件级存储（用户、全局）会自动将 Python 基本类型（str、int、dict、list 等）序列化为 JSON 格式存储，不支持的类型（如对象、函数）需手动转换。

三、app.storage 的基础使用方法

`app.storage` 的所有存储层均提供**字典式操作接口**，核心操作包括数据的增删改查、清空、判断键存在等，使用方式与 Python 原生字典高度一致。

3.1 会话存储（app.storage.session）

会话存储是**内存级、客户端隔离**的临时存储，与 `ui.session` 深度联动，日常开发中优先使用 `ui.session`（上下文友好封装），仅在跨上下文访问时使用 `app.storage.session`。

```
from nicegui import ui, app

@ui.page('/')
def index():
    # 1. 通过会话 ID 访问当前客户端的会话存储
    session_id = ui.session.id
    # 写入数据：键为任意字符串，值为 Python 基本类型
    app.storage.session[session_id]['temp_filter'] = {'keyword': 'nicegui', 'page': 1}
    app.storage.session[session_id]['visit_count'] = 0

    # 2. 读取数据：使用 get() 避免 KeyError
    filter_data = app.storage.session[session_id].get('temp_filter', {})
    visit_count = app.storage.session[session_id].get('visit_count', 0)

    ui.label(f"临时筛选条件: {filter_data}")
    ui.label(f"访问次数: {visit_count}")

    # 3. 更新数据
    def increase_visit():
        app.storage.session[session_id]['visit_count'] += 1
        ui.label(f"更新后访问次数: {app.storage.session[session_id]['visit_count']}")

    # 4. 删除数据
    def clear_filter():
        app.storage.session[session_id].pop('temp_filter', None)
        ui.notify('临时筛选条件已清除', type='warning')

    ui.button('增加访问次数', on_click=increase_visit)
    ui.button('清除筛选条件', on_click=clear_filter)

if __name__ in {'__main__', '__mp_main__'}:
```

```
ui.run()
```

关键说明：`app.storage.session` 是一个**嵌套字典**，外层键为会话 ID，内层为具体的键值对数据；`ui.session` 本质是 `app.storage.session[ui.session.id]` 的上下文封装，因此 `ui.session['key'] = value` 等价于 `app.storage.session[ui.session.id]['key'] = value`。

3.2 用户存储 (app.storage.user)

用户存储是**文件级、客户端隔离**的持久化存储，数据自动保存到 JSON 文件，服务重启后仍可恢复，是实现“记住我”、用户偏好设置的核心。

```
from nicegui import ui, app

@ui.page('/')
def index():
    # 1. 写入用户存储：直接通过键值对赋值，自动持久化
    app.storage.user['username'] = '张三'
    app.storage.user['theme'] = 'dark'
    app.storage.user['settings'] = {'notifications': True, 'font_size': 14}

    # 2. 读取用户存储：使用 get() 提供默认值
    username = app.storage.user.get('username', '游客')
    theme = app.storage.user.get('theme', 'light')
    settings = app.storage.user.get('settings', {})

    ui.label(f"当前用户: {username}").classes('text-2xl')
    ui.label(f"主题设置: {theme}")
    ui.label(f"通知开关: {settings.get('notifications', False)}")

    # 3. 清空用户存储：删除当前用户的所有持久化数据
    def clear_user_storage():
        app.storage.user.clear()
        ui.notify('用户配置已清空', type='error')
        ui.navigate.to('/') # 刷新页面

    ui.button('清空用户配置', on_click=clear_user_storage).classes('mt-4')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```

关键说明：`app.storage.user` 无需手动指定用户标识，框架会自动根据当前会话的**用户标识**（默认会话 ID）关联到对应的数据集，因此不同客户端的 `app.storage.user` 操作的是各自独立的持久化数据。

3.3 全局存储 (app.storage.general)

全局存储是**文件级、应用全局共享**的持久化存储，所有客户端访问同一数据集，适用于存储应用的全局参数。

```
from nicegui import ui, app

@ui.page('/')
def index():
```

```

# 1. 初始化全局存储：仅在首次运行时设置默认值
if 'app_version' not in app.storage.general:
    app.storage.general['app_version'] = '1.0.0'
if 'max_online_users' not in app.storage.general:
    app.storage.general['max_online_users'] = 0
if 'online_users' not in app.storage.general:
    app.storage.general['online_users'] = []

# 2. 读取全局存储
version = app.storage.general['app_version']
max_users = app.storage.general['max_online_users']
online_users = app.storage.general['online_users']

ui.label(f"应用版本: {version}").classes('text-2x1')
ui.label(f"最大在线用户数: {max_users}")
ui.label(f"当前在线用户: {len(online_users)}").classes('text-green-500')

# 3. 更新全局存储：模拟用户上线
def user_online():
    user_id = ui.session.id[:8] # 取会话 ID 的前 8 位作为用户标识
    if user_id not in online_users:
        app.storage.general['online_users'].append(user_id)
        # 更新最大在线用户数
        current_count = len(app.storage.general['online_users'])
        if current_count > app.storage.general['max_online_users']:
            app.storage.general['max_online_users'] = current_count
    ui.navigate.to('/')

ui.button('模拟用户上线', on_click=user_online).classes('mt-4')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()

```

关键说明： `app.storage.general` 的数据对所有客户端可见，修改后会实时同步到所有访问的客户端；适合存储无需隔离的应用级参数，如版本号、全局开关、系统配置等。

四、app.storage 的持久化机制

`app.storage` 的文件级存储（`user`、`general`）基于 **JSON 文件** 实现持久化，其底层机制包括**存储路径**、**自动序列化 / 反序列化**、**数据刷新**三个核心环节。

4.1 存储路径

NiceGUI 默认将 `app.storage` 的持久化数据存储在**用户主目录**的 `.nicegui/storage` 文件夹中，不同操作系统的路径如下：

- Windows: `C:\Users\<用户名>\.nicegui\storage`
- macOS/Linux: `~/ .nicegui/storage`

其中：

- `user` 存储层的数据保存在 `storage/users` 目录下，每个用户标识对应一个 JSON 文件；

- `general` 存储层的数据保存在 `storage/general.json` 文件中。

自定义存储路径

开发者可通过修改 `app.storage.path` 自定义存储根路径，建议在应用启动前配置：

```
from nicegui import app
from pathlib import Path

# 自定义存储路径为应用根目录下的 storage 文件夹
app.storage.path = Path(__file__).parent / 'storage'
```

4.2 自动序列化 / 反序列化

`app.storage` 会自动将 Python 数据类型**序列化为 JSON 格式**写入文件，读取时再**反序列化为 Python 类型**，支持的基础类型包括：

- 字符串 (str)、数字 (int/float)、布尔值 (bool)、None；
- 列表 (list)、字典 (dict)（键需为字符串）。

不支持的类型：对象 (class instance)、函数 (function)、集合 (set)、日期 (datetime) 等。若需存储这些类型，需手动转换为基础类型（如将 datetime 转为字符串，将对象转为字典）。

示例：存储 datetime 类型

```
from nicegui import ui, app
from datetime import datetime

@ui.page('/')
def index():
    # 将 datetime 转为字符串存储
    app.storage.user['last_login'] = datetime.now().isoformat()
    # 读取后转换回 datetime
    last_login_str = app.storage.user.get('last_login')
    if last_login_str:
        last_login = datetime.fromisoformat(last_login_str)
        ui.label(f"最后登录时间: {last_login.strftime('%Y-%m-%d %H:%M:%S')}")

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```

4.3 数据刷新机制

`app.storage` 的文件级存储采用**惰性写入 + 实时读取**的机制：

1. **写入：**对 `app.storage.user` / `app.storage.general` 的数据修改会**立即写入内存**，并在短时间内**异步写入文件**（避免频繁 IO 操作）；
2. **读取：**每次访问数据时，优先从内存缓存读取，若文件有更新（如其他进程修改），会自动重新加载文件数据；
3. **持久化保障：**应用正常退出时，框架会强制将内存中的缓存数据写入文件，确保数据不丢失。

五、app.storage 的高级使用场景

结合 NiceGUI 的会话管理、鉴权、生命周期等特性，`app.storage` 可实现更复杂的业务需求，以下是典型的高级场景。

5.1 实现“记住我”功能

利用 `app.storage.user` 的持久化特性，实现登录状态的长期保留，即使服务重启或客户端关闭浏览器，再次访问时仍可自动登录。

```
from nicegui import ui, app

# 模拟用户数据库
USER_DB = {
    "admin": {"password": "admin123", "role": "admin"},
    "user": {"password": "user123", "role": "user"}
}

# 鉴权装饰器
def require_login(page):
    def wrapper():
        # 优先从会话读取登录状态，若无则从用户存储恢复
        if not ui.session.get('is_login'):
            if app.storage.user.get('remember_me'):
                # 从用户存储恢复登录状态
                ui.session['is_login'] = True
                ui.session['username'] = app.storage.user['username']
                ui.session['role'] = app.storage.user['role']
            else:
                ui.navigate.to('/login')
        return
    return page()
return wrapper

@ui.page('/login')
def login_page():
    username_input = ui.input('用户名')
    password_input = ui.input('密码').props('type=password')
    remember_me = ui.checkbox('记住我')

    def do_login():
        username = username_input.value.strip()
        password = password_input.value.strip()
        user = USER_DB.get(username)
        if not user or user['password'] != password:
            ui.notify('账号或密码错误', type='error')
            return
        # 更新会话状态
        ui.session['is_login'] = True
        ui.session['username'] = username
        ui.session['role'] = user['role']
        # 若勾选“记住我”，则持久化到用户存储
```

```

        if remember_me.value:
            app.storage.user['remember_me'] = True
            app.storage.user['username'] = username
            app.storage.user['role'] = user['role']
        else:
            # 清除记住我状态
            app.storage.user.pop('remember_me', None)
            ui.navigate.to('/dashboard')

    ui.button('登录', on_click=do_login)

@ui.page('/dashboard')
@require_login
def dashboard():
    ui.label(f"欢迎回来, {ui.session['username']}!")
    def do_logout():
        # 清除会话状态和用户存储的记住我状态
        ui.session.clear()
        app.storage.user.pop('remember_me', None)
        ui.navigate.to('/login')
    ui.button('登出', on_click=do_logout)

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()

```

5.2 跨会话的用户配置同步

利用 `app.storage.user` 的持久化特性，实现用户配置在不同设备 / 浏览器中的同步（需结合用户账号体系，替换默认的用户标识）。

```

from nicegui import ui, app

# 模拟用户登录后，将用户标识从会话 ID 改为用户 ID
def set_user_identifier(user_id: str):
    # 切换用户存储的标识（需结合实际用户体系实现）
    app.storage.set_user_identifier(user_id)

@ui.page('/')
def index():
    # 模拟登录：用户 ID 为 1001
    set_user_identifier('1001')
    # 存储用户配置（绑定到用户 ID 1001，而非会话 ID）
    app.storage.user['theme'] = 'dark'
    app.storage.user['font_size'] = 16
    # 读取配置
    ui.label(f"用户 1001 的主题: {app.storage.user['theme']}")
    ui.label(f"用户 1001 的字体大小: {app.storage.user['font_size']}")

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()

```

关键说明： `app.storage.set_user_identifier()` 是 NiceGUI 提供的切换用户标识的接口，结合实际的用户账号体系（如用户 ID、手机号），可实现同一用户在不同设备上的配置同步。

5.3 应用全局配置的持久化

利用 `app.storage.general` 存储应用的全局配置，实现配置的持久化和动态更新，无需修改代码即可调整应用行为。

```
from nicegui import ui, app

# 应用启动时初始化全局配置
@app.on_startup
async def init_config():
    if 'site_title' not in app.storage.general:
        app.storage.general['site_title'] = 'NiceGUI 应用'
    if 'enable_registration' not in app.storage.general:
        app.storage.general['enable_registration'] = True
    if 'max_upload_size' not in app.storage.general:
        app.storage.general['max_upload_size'] = 10 # MB

@ui.page('/')
def index():
    # 读取全局配置
    site_title = app.storage.general['site_title']
    enable_reg = app.storage.general['enable_registration']
    max_upload = app.storage.general['max_upload_size']

    ui.label(site_title).classes('text-3xl font-bold')
    ui.label(f"是否开启注册: {'是' if enable_reg else '否'}")
    ui.label(f"最大上传文件大小: {max_upload} MB")

    # 动态修改全局配置
    def update_config():
        app.storage.general['site_title'] = '新的应用标题'
        app.storage.general['enable_registration'] = False
        ui.notify('全局配置已更新', type='success')
        ui.navigate.to('/')

    ui.button('修改全局配置', on_click=update_config).classes('mt-4')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```

六、app.storage 的注意事项与常见坑

6.1 不支持的存储类型

`app.storage` 的文件级存储仅支持 JSON 可序列化的类型，若存储不支持的类型（如对象、集合、datetime），会抛出序列化异常。

解决方案： 手动将非基础类型转换为基础类型（如将对象转为字典，将 datetime 转为 ISO 字符串）。

6.2 数据一致性问题

多进程 / 多实例部署时，`app.storage` 的文件级存储可能出现**数据一致性问题**（多个进程同时读写同一文件）。

解决方案：

- 单进程部署：适合小型应用，无需额外处理；
- 多进程 / 分布式部署：替换为 Redis、MySQL 等分布式存储，而非 `app.storage` 的文件存储。

6.3 存储性能问题

`app.storage` 的文件级存储基于本地 JSON 文件，频繁的大数量级数据操作会导致 IO 性能瓶颈。

解决方案：

- 仅存储**轻量级数据**（配置、状态标识等），不存储大型数据集；
- 大型数据集使用数据库（SQLite/MySQL/PostgreSQL）存储，`app.storage` 仅保存数据索引。

6.4 用户标识的默认行为

`app.storage.user` 默认使用会话 ID 作为用户标识，若客户端清除 Cookie（会话 ID 丢失），则无法访问原有的用户存储数据。

解决方案：

- 结合用户账号体系，使用**用户 ID**替代会话 ID 作为用户标识；
- 提供“绑定账号”功能，将临时的会话标识与永久的用户标识关联。

6.5 数据备份与迁移

`app.storage` 的持久化数据存储在本地图文件中，需注意数据备份与迁移。

解决方案：

- 定期备份 `app.storage.path` 目录下的文件；
- 迁移时直接复制存储目录到新服务器，修改 `app.storage.path` 指向新路径即可。

七、app.storage 与 ui.session 的关系

`app.storage` 与 `ui.session` 是 NiceGUI 中紧密关联的两个工具，二者的核心关系可总结为：

1. **封装与底层**：`ui.session` 是 `app.storage.session[ui.session.id]` 的**上下文友好封装**，日常开发中优先使用 `ui.session` 操作会话数据，仅在跨上下文访问时使用 `app.storage.session`；
2. **临时与持久**：`ui.session`（`app.storage.session`）是内存临时存储，服务重启后丢失；`app.storage.user / general` 是文件持久化存储，数据长期保留；
3. **互补使用**：结合 `ui.session` 存储临时状态，`app.storage.user` 存储持久化配置，`app.storage.general` 存储全局参数，可满足绝大多数场景的存储需求。

八、总结

`app.storage` 是 NiceGUI 提供的一站式分层存储解决方案，通过会话存储、用户存储、全局存储三个层级，完美适配了临时状态、持久化配置、全局参数的不同存储需求。其统一的字典式接口降低了开发成本，自动持久化机制简化了文件读写逻辑，与 `ui.session`、`app` 类的深度集成让其成为 NiceGUI 应用开发的核心工具。

掌握 `app.storage` 的使用，可实现：

1. **临时状态管理**：通过会话存储保存客户端的临时数据；
2. **用户配置持久化**：通过用户存储实现“记住我”、个性化设置等功能；
3. **全局参数共享**：通过全局存储实现应用级配置的统一管理；
4. **跨设备配置同步**：结合用户账号体系，实现同一用户的配置同步。

在实际项目中，可根据业务需求选择合适的存储层，对于大型应用或分布式部署，可将 `app.storage` 与数据库、Redis 等存储工具结合，进一步提升数据存储的性能和可靠性。